

FAST National University of Computer and Emerging Sciences

PROJECT REPORT

*An Object-Oriented C++ Application Implementing Abstraction,
Inheritance, Polymorphism, and Static Members*

Course:	Object Oriented Programming (OOP)
Instructor:	Sir Muhammad Khalid Khan
Section:	BAI-2B
Semester:	Spring 2026
Submission Date:	April 2026

Group Members:

Member 1 : Ahmed Asif 25K-0039
Member 2 : Muhammad Ahmed 25K-0105
Member 3 : Ahmad Ali Tariq 25K-0138

Table of Contents

1. Executive Summary	2
2. Introduction	3
2.1 Background & Motivation	3
2.2 Problem Statement	3
2.3 Objectives	3
2.4 Scope	3
3. System Analysis	3
3.1 Functional Requirements	3
3.2 Non-Functional Requirements	4
4. Object-Oriented Design	4
4.1 Class Hierarchy & UML Diagram	4
4.2 Class Descriptions	4
4.3 Relationships Between Classes	4
5. Implementation	5
5.1 OOP Concepts Implemented	5
5.2 Code Snippets	6
6. Testing & Results	6
7. Challenges & Solutions	6
8. Division of Work	7
9. Conclusion & Future Work	7
10. References	7

1. Executive Summary

This report presents the complete design and implementation of the **Quiz Application**, a C++ console-based software developed to facilitate automated testing and classroom management. Developed for the Object Oriented Programming (OOP) Lab at FAST-NUCES Karachi, the system allows teachers to register, create virtual classrooms, and design time-limited quizzes. Students can join these classes and attempt quizzes, with the system automatically calculating scores and maintaining a real-time leaderboard.

The application serves as a practical demonstration of core OOP paradigms, including encapsulation, inheritance, and abstraction. Key features include an automated ID generation system using static members, polymorphic user handling through an abstract base class, and secure classroom access via password protection.

2. Introduction

2.1 Background & Motivation

In modern educational environments, manual assessment is often time-consuming and difficult to track. The transition to digital platforms requires robust systems that can handle multiple user roles (Teachers and Students) and enforce strict rules such as time limits and enrollment constraints. The motivation for this project was to build a self-contained system that simulates these real-world classroom interactions using Object-Oriented principles.

2.2 Problem Statement

The core challenges addressed by this system include:

- Lack of a centralized platform for managing classroom-specific assessments.
- Inability to enforce strict time limits on manual quiz attempts.
- Difficulty in tracking student performance and ranking across different subjects.
- Managing different permissions for instructors versus participants.

2.3 Objectives

1. Implement a secure multi-user system using **Inheritance** and **Abstraction**.
2. Create a **Classroom-Quiz relationship** using aggregation and composition.
3. Develop a **real-time timer** for quiz attempts using the `<ctime>` library.
4. Provide automated performance feedback and leaderboard generation.

3. System Analysis

3.1 Functional Requirements

ID	Requirement	Priority
FR-01	Teacher can register and receive a unique 'F' prefix ID.	High
FR-02	Teacher can create classrooms with unique passwords.	High
FR-03	Teacher can add quizzes with specific time limits to classrooms.	High
FR-04	Student can join classrooms using a Class ID and Password.	High
FR-05	System enforces a timer; quiz ends automatically if time expires.	High
FR-06	System calculates marks and updates the student's cumulative score.	Medium

FR-07	Leaderboard displays students ranked by their total scores.	Medium
-------	---	--------

3.2 Non-Functional Requirements

- **Memory Management:** All dynamic memory (newly created Teachers, Students, Classrooms) is tracked and deleted in the System destructor.
- **Security:** Actions like joining a class or creating a quiz require password verification.
- **Robustness:** Input validation is used to handle invalid quiz choices or numeric inputs.

4. Object-Oriented Design

4.1 Class Hierarchy

The system is built around the following structure:

- **User (Abstract Base):** Defines common attributes (name, id, password) and the pure virtual `setuser()` method.
- **Teacher & Student:** Derived from `User`. `Teacher` manages classrooms; `Student` attempts quizzes.
- **Classroom:** Aggregates a `Teacher` and a collection of `Students`. It contains a vector of `Quiz` objects.
- **Quiz:** Contains a vector of `Question` objects and manages time limits.
- **Attempt:** A temporary object that links a `Student` to a `Quiz` to handle the logic of the exam.
- **System:** The central controller managing all vectors of users and classrooms.

4.2 Relationships Between Classes

Relationship	Classes Involved	Type
User is base of Teacher/Student	User, Teacher, Student	Inheritance (IS-A)
Classroom contains Quizzes	Classroom, Quiz	Composition
Classroom manages Students	Classroom, Student	Aggregation
System manages all entities	System, Teacher, Student, Class	Association
Attempt links Student and Quiz	Attempt, Student, Quiz	Dependency

5. Implementation

5.1 OOP Concepts Implemented

Concept	Implementation Detail
Abstraction	User is an abstract class with a pure virtual <code>setuser</code> function.
Inheritance	Teacher and Student inherit from User to reuse ID and name properties.
Encapsulation	Private members like <code>correctoption</code> in Question are protected from direct access.
Static Members	<code>idn</code> and <code>idnum</code> are static ints used to generate unique serial IDs.
Friend Classes	System is a friend of Teacher and Student to access IDs and passwords for validation.
Polymorphism	Virtual destructors and method overriding in derived user classes.

5.2 Code Snippet: User Hierarchy

```
class User {
protected:
    string name;
    string id;
    int password;
public:
    User(string n = "xxx", int i = 0) : name(n), password(i) {}
    virtual void setuser(string s) = 0; // Pure Virtual
    virtual ~User() {}
};

class Teacher : public User {
    static int idn;
public:
    Teacher(string n, int pass) : User(n, pass) {
        id = "F" + std::to_string(idn++);
    }
    void setuser(string s) override { name = s; }
};
```

5.3 Code Snippet: Timer Logic

```
bool istimeover() {
    time_t now = time(0);
    return difftime(now, starttime) >= quiz->timelim;
}

// Inside startattempt loop:
if (istimeover()) {
    cout << "Time UP!" << endl;
    break;
}
```

6. Testing & Results

The system underwent several functional tests to ensure the logic remained consistent across different user roles.

- Leaderboard Sort

Test Case	Input	Expected Result	Status
User Registration	Name, Password	Unique ID (e.g., S0, F0) generated.	Pass
Classroom Access	Wrong Password	"Wrong Password!" error displayed.	Pass
Quiz Attempt	Timed Input	If time > limit, attempt terminates.	Pass
Multiple Students	Students sorted by descending scores.	Pass	

7. Challenges & Solutions

- **Challenge:** Handling the quiz timer in a console application without multi-threading.
Solution: Used the `<ctime>` library to check the difference between the start time and the current time at every question prompt.
- **Challenge:** Cross-class data access (e.g., System needs to check Student IDs).
Solution: Implemented `friend class` relationships to allow the management system to verify credentials securely.

8. Division of Work

Member	Responsibilities	Contribution
Member 1 (Lead)	Class hierarchy, User/Teacher/Student inheritance, ID generation logic, and Polymorphism.	34%
Member 2	Quiz logic, Question validation, Attempt timer implementation using ctime, and Leaderboard sorting.	33%
Member 3	Classroom/System management classes, main menu interface, and project documentation.	33%

9. Conclusion & Future Work

The Quiz Application successfully demonstrates how C++ Object-Oriented principles can be used to build a complex, multi-entity system. By utilizing inheritance and friend classes, the project maintains a clean separation of concerns between the user interface and the underlying data.

Future Work:

- Implementing File I/O (fstream) to save user and score data permanently.
- Adding different question types (True/False, Multiple Response).
- Developing a graphical interface using SFML or Qt for a better user experience.

10. References

1. Stroustrup, B. The C++ Programming Language, 4th Edition. Addison Wesley, 2013.
2. Lippman, S., Lajoie, J., Moo, B. C++ Primer, 5th Edition. Addison-Wesley, 2012.
3. cppreference.com. C++ Reference — Classes, Templates, STL. <https://en.cppreference.com/>.
4. GeeksforGeeks. Object Oriented Programming in C++. <https://www.geeksforgeeks.org/oop-in-cpp/>.
5. LearnCpp.com. Virtual Functions, Inheritance, Templates. <https://www.learncpp.com/>.
6. ISO/IEC 14882:2017. Programming Language — C++ (C++17 Standard).
7. Course Lab Materials — Sir Muhammad Khalid Khan, FAST-NUCES Karachi, OOP Lab, Spring 2026.